**Week 2 – Day 1 – Assignment**

# Automation Framework Using Chain-of-Thoughts method:-

Context:- -As a test lead give me a mobile testing framework for regression suite
Instruction:-
-Framework should include reporting and CI/CD integration
-[IMPORTANT] Ask me 10 questions on this and follow step by step procedure once after proceeding with next question

Got it — we'll design a **Mobile Regression Testing Framework (Test Lead level)** with **CI/CD integration + reporting**, but I'll first gather requirements step-by-step.

---

## ✅ Step 1 – Question 1 of 10

**Q1. What type of mobile application is this framework for?**

(e.g., Banking, E-commerce, Healthcare, Social Media, Enterprise, Gaming)

👉 This impacts:

- regression risk areas
- security depth
- automation priority

Please answer, and I'll proceed to **Question 2**.

Automation Priority

I think there's a slight mismatch in the answer 😀 — I asked for the **type of mobile application**, not the automation priority.

Let's correct that so I can design the framework properly.

---

## ✅ Step 1 – Question 1 of 10 (Re-asked)

**Q1. What type of mobile application is this regression framework for?**

Examples:

- Banking
- E-commerce
- Healthcare
- Social media
- Enterprise/internal app
- Gaming
- Other (please specify)

👉 This helps define:

- regression scope depth
- test prioritization
- CI/CD quality gates

Please answer this, and I'll move to **Question 2**.

Banking

Great — **Banking application** confirmed 👍 (high reliability + high security regression focus)

---

## ✅ Step 2 – Question 2 of 10

**Q2. Which mobile platforms must the regression framework support?**

- Android only
- iOS only
- Both Android and iOS
- Also include tablets (iPad / Android tablets)

👉 This impacts:

- device matrix strategy
- parallel execution in CI/CD
- automation framework design (Appium vs native tools)

Please answer, and I'll proceed to **Question 3**.

Both Android and iOS

Got it — **Android + iOS** support confirmed 👍

---

## ✅ Step 3 – Question 3 of 10

**Q3. What automation tools or tech stack are preferred (if any)?**

Choose one or more:

- Appium (cross-platform UI automation)
- Espresso (Android native)
- XCUITest (iOS native)
- Selenium-based mobile wrapper
- Not decided / need recommendation

👉 This defines:

- framework architecture
- reuse strategy across platforms
- CI/CD integration approach

Please answer, and I'll proceed to **Question 4**.

Appium (cross-platform UI automation)

Perfect — **Appium (cross-platform UI automation)** confirmed 👍
This fits well for a unified regression framework across Android + iOS.

---

## ✅ Step 4 – Question 4 of 10

**Q4. What level of test automation coverage do you expect for the regression suite?**

- Smoke only (10–20%)
- Partial regression (40–60%)
- High coverage regression (70–90%)
- Nearly full automation (90–100%)

👉 This will determine:

- framework complexity
- CI/CD execution time strategy
- parallelization and tagging design

Please answer, and I'll proceed to **Question 5**.

Nearly full automation (90–100%)

Great — **90–100% regression automation** confirmed 👍
This means we'll design a **highly scalable, CI/CD-first Appium framework with strong stability controls and parallel execution**.

---

## ✅ Step 5 – Question 5 of 10

**Q5. What CI/CD tools are used or preferred for pipeline integration?**

Choose one or more:

- Jenkins
- GitHub Actions
- GitLab CI
- Azure DevOps
- Bitbucket Pipelines
- Not decided / need recommendation

👉 This impacts:

- pipeline design
- reporting integration
- test triggering strategy (PR, nightly, release builds)

Please answer, and I'll proceed to **Question 6**.

Jenkins

Great — **Jenkins CI/CD** confirmed 👍
We'll design a pipeline-driven regression framework with Jenkins stages, parallel execution, and automated reporting.

---

## ✅ Step 6 – Question 6 of 10

**Q6. What types of regression tests should be included in the framework?**

Select all that apply:

- UI functional regression (end-to-end user flows)
- API regression tests
- Backend integration validation
- Database validation checks
- Visual regression testing
- Network / offline scenario tests

👉 This defines:

- framework layers
- tool integration strategy
- execution grouping in CI/CD

Please answer, and I'll proceed to **Question 7**.

UI functional regression (end-to-end user flows
API regression tests
Database validation checks

Great — we now have a **3-layer regression scope** confirmed 👍
(UI + API + DB validation = strong end-to-end quality gate)

---

## ✅ Step 7 – Question 7 of 10

**Q7. What is the expected execution strategy for the regression suite in CI/CD (Jenkins)?**

Choose one:

- On every code commit (very fast smoke-style runs)
- On pull request (PR) merge validation
- Nightly scheduled runs
- Pre-release / staging deployment only
- Hybrid (smoke on PR + full regression nightly + pre-release suite)

👉 This determines:

- pipeline stages
- test suite segmentation (smoke/regression/full)
- execution frequency and resource scaling

Please answer, and I'll proceed to **Question 8**.

Hybrid (smoke on PR + full regression nightly + pre-release suite)

Perfect — **Hybrid execution model (PR smoke + nightly full regression + pre-release suite)** is exactly what a mature banking CI/CD setup needs 👍

This gives us:

- Fast feedback on PRs
- Stable nightly confidence runs
- Strict release gating before production

---

## ✅ Step 8 – Question 8 of 10

**Q8. What reporting expectations do you have for the regression framework?**

Choose all that apply:

- HTML test execution reports (local / CI artifacts)
- Jenkins dashboards (build-wise visibility)
- Email notifications (pass/fail summaries)
- Slack/Teams integration alerts
- Detailed defect linking (Jira integration)
- Historical trend reports (pass rate, flakiness, execution time)

👉 This defines:

- reporting architecture
- CI visibility strategy
- stakeholder communication flow

Please answer, and I'll proceed to **Question 9**

HTML test execution reports (local / CI artifacts)
Detailed defect linking (Jira integration)
Historical trend reports (pass rate, flakiness, execution time)

Great — this is a **strong enterprise-grade reporting setup** 👍
We'll design:

- CI artifacts for debugging
- Jira-linked defect traceability
- trend analytics for regression health + flakiness control

---

## ✅ Step 9 – Question 9 of 10

**Q9. What environment setup should the regression framework support?**

Choose one or more:

- Dev environment
- QA/Test environment
- UAT environment
- Dedicated automation environment (recommended for stability)
- Device cloud (BrowserStack / Sauce Labs / LambdaTest)
- On-prem real device lab

👉 This defines:

- execution stability strategy
- device provisioning model
- parallel execution scalability

Please answer, and I'll proceed to the final Question 10.

Dev environment
QA/Test environment
UAT environment

Perfect — **Dev + QA/Test + UAT environments** confirmed 👍
This means we'll design the framework with **environment-aware configuration, data isolation, and promotion-based execution gates**.

---

## ✅ Step 10 – Question 10 of 10

**Q10. What is the preferred test data strategy for regression automation?**

Choose one or more:

- Static test data (predefined accounts/users)
- Dynamic test data generation (created during runtime)
- API-driven test data setup (via backend services)
- Database seeding scripts
- Shared centralized test data environment
- Masked production-like data

👉 This is critical for:

- regression stability
- data dependency reduction
- CI/CD reliability (avoiding flaky tests)

Please answer, and once you respond I will generate your **complete Mobile Regression Testing Framework (Appium + Jenkins + Reporting + Architecture + Execution Strategy)**.

# 📘 MOBILE REGRESSION TESTING FRAMEWORK (TEST LEAD DESIGN)

## 🏦 Application Context

- Domain: **Banking**
- Platforms: **Android + iOS**
- Automation Tool: **Appium (Cross-platform)**
- CI/CD: **Jenkins**
- Coverage: **UI + API + DB validation**
- Execution Model: **Hybrid (PR Smoke + Nightly Full + Pre-release suite)**
- Environments: **Dev, QA/Test, UAT**
- Test Data Strategy: **Static + Dynamic + API-driven**

---

# 1. 🏗️ FRAMEWORK ARCHITECTURE

## 1.1 High-Level Structure

```
Mobile Regression Framework
│
├── UI Layer (Appium)
│   ├── Page Object Model (POM)
│   ├── Screen Actions
│   └── Assertions
│
├── API Layer
│   ├── REST Assured / HTTP Client
│   ├── Auth Token Manager
│   └── Service Validators
│
```

```
│
├── Database Layer
│   ├── JDBC / DB connectors
│   ├── Data validation queries
│   └── Reconciliation checks
│
├── Test Layer
│   ├── Smoke Suite (PR)
│   ├── Regression Suite (Nightly)
│   ├── Pre-release Suite
│
├── Test Data Layer
│   ├── Static dataset (accounts/users)
│   ├── Dynamic data generator
│   └── API data provisioning
│
└── Reporting Layer
    ├── HTML reports
    ├── Jenkins reports
    ├── Jira integration
    └── Trend analytics
```

## 2. 🧪 REGRESSION SUITE DESIGN

### 2.1 Suite Segmentation

#### 🔹 Smoke Suite (PR Validation)

- Login (PIN authentication)
- Account balance fetch
- Basic fund transfer (mocked/limited)
- API health checks

👉 Goal: **Fast feedback (<10–15 min execution)**

---

#### 🔹 Full Regression Suite (Nightly)

- Login scenarios (positive/negative)
- Fund transfer (NEFT/IMPS/UPI flows)
- Statement generation
- API validation for all endpoints
- DB reconciliation checks
- Session handling & timeout
- Negative and edge cases

👉 Goal: **Full system validation**

---

#### 🔹 Pre-Release Suite

- End-to-end business flows
- Cross-module validation
- Full API + DB reconciliation
- Security validation regression checks
- Data consistency checks across systems

👉 Goal: **Go/No-Go release decision**

# 3. 🔗 CI/CD INTEGRATION (JENKINS)

## 3.1 Pipeline Design

```
Jenkins Pipeline Stages
|
├── Code Checkout (Git)
├── Build & Dependency Install
├── Environment Setup (Dev / QA / UAT)
├── Deploy Test App (APK / IPA)
├── Start Appium Server
├── Execute Smoke / Regression / Pre-release Suite
├── Generate Reports
├── Publish Results (HTML + Jenkins)
├── Push Results to Jira
└── Notify Teams (Email / Slack optional)
```

---

## 3.2 Execution Strategy

| Trigger | Suite |
|---|---|
| PR Merge | Smoke Suite |
| Nightly Schedule | Full Regression Suite |
| Pre-release Deployment | Pre-release Suite |

---

## 3.3 Parallel Execution Strategy

- Device-level parallel execution
- Test class-level parallelism (TestNG/JUnit)
- Appium Grid or device farm integration ready

# 4. 📊 REPORTING FRAMEWORK

## 4.1 HTML Reports

- TestNG / Allure-style reports
- Execution summary
- Step-level logs
- Screenshot on failure

---

## 4.2 Jenkins Dashboard

- Build-wise execution status
- Pass/fail metrics
- Trend visualization plugins

---

## 4.3 Jira Integration

- Auto defect creation on failure
- Link test cases → Jira stories
- Attach logs + screenshots

---

## 4.4 Trend Analytics

Tracked metrics:

- Pass rate over time
- Flaky test identification
- Execution duration trends
- Environment stability score

---

## 5. 🧮 APP BEHAVIOR VALIDATION (REGRESSION FOCUS)

### 5.1 Login (PIN-Based)

- Max retry lock validation
- Session timeout enforcement
- Secure input handling
- Invalid PIN error handling

---

### 5.2 Fund Transfer

- Transaction success/failure validation
- Duplicate transaction prevention
- API response validation
- DB transaction consistency check

---

### 5.3 Account Services

- Balance accuracy validation (UI vs API vs DB)
- Statement correctness
- Real-time sync validation

---

### 5.4 Network Behavior (Critical)

- Offline handling
- API timeout recovery
- Retry mechanism validation

---

### 5.5 Data Integrity

- UI ↔ API ↔ DB consistency checks
- Transaction reconciliation validation

## 6. 📄 TEST DATA STRATEGY

### 6.1 Static Test Data

- Predefined user accounts
- Fixed test bank accounts
- Controlled transaction datasets

---

### 6.2 Dynamic Test Data

- Runtime generated user transactions
- Randomized transfer amounts
- Temporary session accounts

---

### 6.3 API-driven Data Setup

- Create users/accounts via backend APIs
- Reset test states before execution
- Data cleanup after test runs

---

## 7. ⚠️ KEY FRAMEWORK FEATURES

### ✔ Stability Design

- Retry mechanism for flaky UI tests
- Smart waits (not hard sleeps)
- Network-aware validations

### ✔ Maintainability

- Page Object Model (POM)
- Modular test design
- Reusable API utilities

### ✔ Scalability

- Parallel execution support
- CI/CD friendly suite tagging
- Device grid ready architecture

---

## 8. 🚀 FINAL SUMMARY

### 🧩 Framework Stack

- Appium (UI automation)
- REST API automation layer
- DB validation layer
- Jenkins CI/CD pipeline
- HTML + Jira + Trend reporting

---

### 🔄 Execution Model

- PR → Smoke tests
- Nightly → Full regression
- Pre-release → Business validation suite

---

### 🏦 Banking Focus

- RBI-aligned security validation
- Strong transaction integrity checks
- Multi-layer verification (UI + API + DB)

---

If you want next, I can also design:

✔ **Folder structure + coding framework (real Appium project structure)**
✔ **Jenkinsfile (ready-to-use pipeline script)**
✔ **Test case design for regression suite (banking flows)**
✔ **Automation framework architecture diagram (v ↓ l)**